

CE 4011 Static and Modal Structural Analysis

Desktop MVP

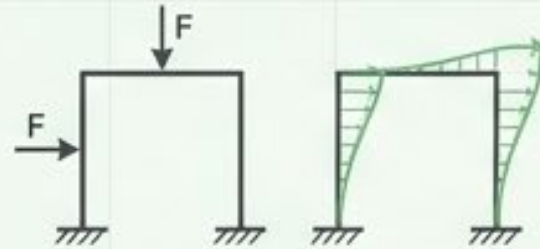
Final Project Report
Mohammad Umair Naeem | ID: 2416055
Spring Semester 2025-2026

Prioritizing Educational Transparency Over Black-Box Systems

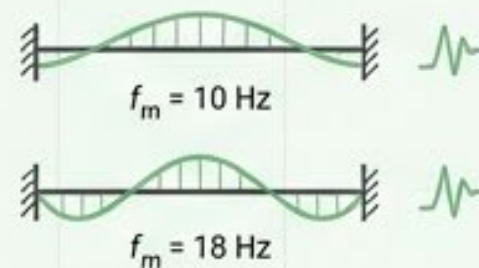
Objective: Educational Python Tkinter application exposing intermediate math (matrices, DOF maps) rather than hiding it behind commercial-scale modeling.

Final Submitted Scope

2D Static Analysis

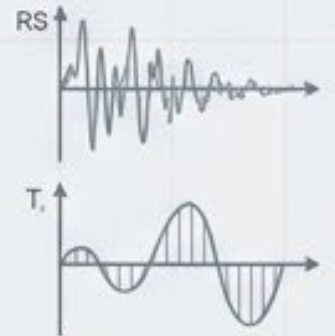


2D Modal Analysis



Deferred & Out of Scope

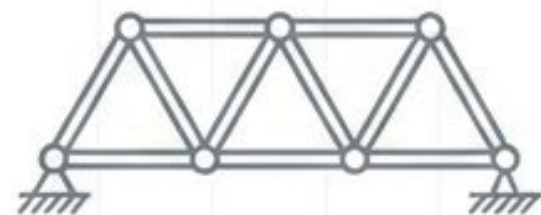
Response Spectrum Analysis (RSA) & Time-History Analysis (THA) [Retained as future backend extensions]



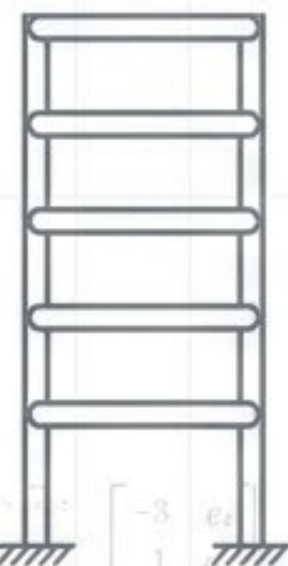
3D framing, non-linear analysis, automated meshing, and design-code checks.



Unified Assembly for Universal 2D Configurations



Supported Systems: 2D Frames, Trusses, Beams, Shear Frames, Cantilevers.



Unified Assembly & Solver Pipeline

Solver Architecture: No branching by structure family. All structures are ordinary configurations of nodes, elements, releases, and masses.

XML
Reproducibility Backend

Educational Results

Reproducibility: Every UI action is backed by an XML save/load/export system.

$$\Rightarrow D_f = \begin{bmatrix} 0 & a_{11} & a_{12} & a_{13} \\ 0 & a_{21} & a_{22} & a_{23} \\ 0 & a_{m1} & a_{m2} & a_{m3} \\ 0 & a_{m1} & a_{m2} & a_{m3} \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & 0 \\ a_{21} & a_{22} & -b_3 \\ -b_2 & a_{m3} & -b_3 \end{bmatrix}$$

$$R_1 = \sum_{i=1}^N [iB_i - Y_0 H(i)]$$

$$D_c = \sum_{i=1}^N [D_i (n - \gamma r_i) + L_i]$$

$$Z_i = Z(x_i, \gamma) \cdot \begin{bmatrix} -3 & e_i \\ 1 & \eta_i \end{bmatrix}$$

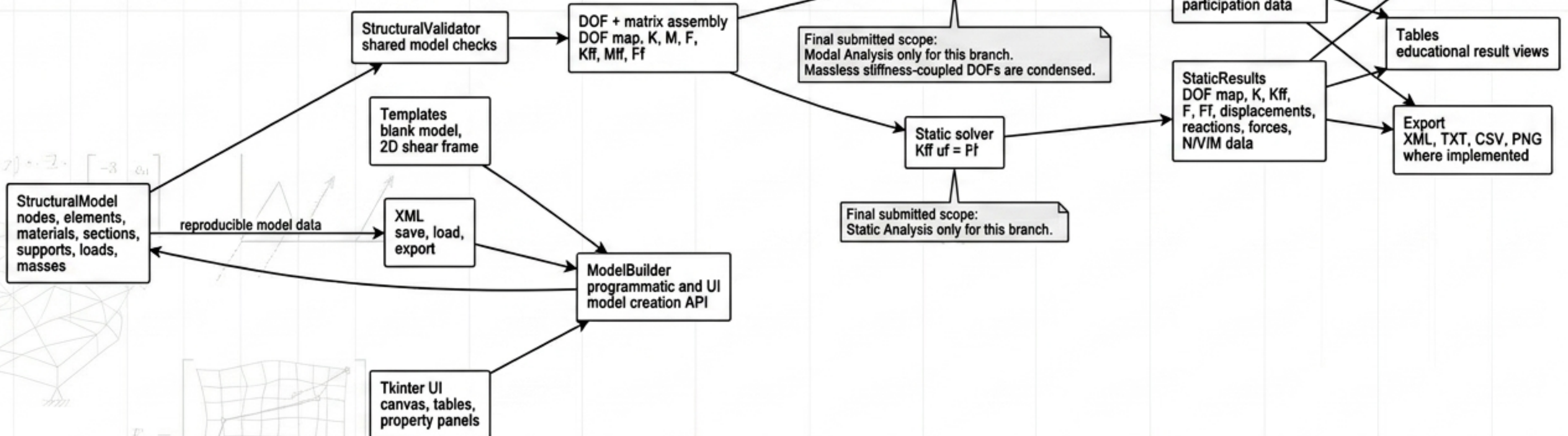
$$\|Displacement\| + \|Velocity\|$$

$$T_{eq} =$$

Strict Object-Oriented Pipeline and Separation of Concerns

Core Principle: Solvers do not touch UI code.

Pipeline Flow: Tkinter Canvas / XML → ModelBuilder → StructuralModel → DOF/K/M/F Assembly → Solver → Result Object → Visualization.

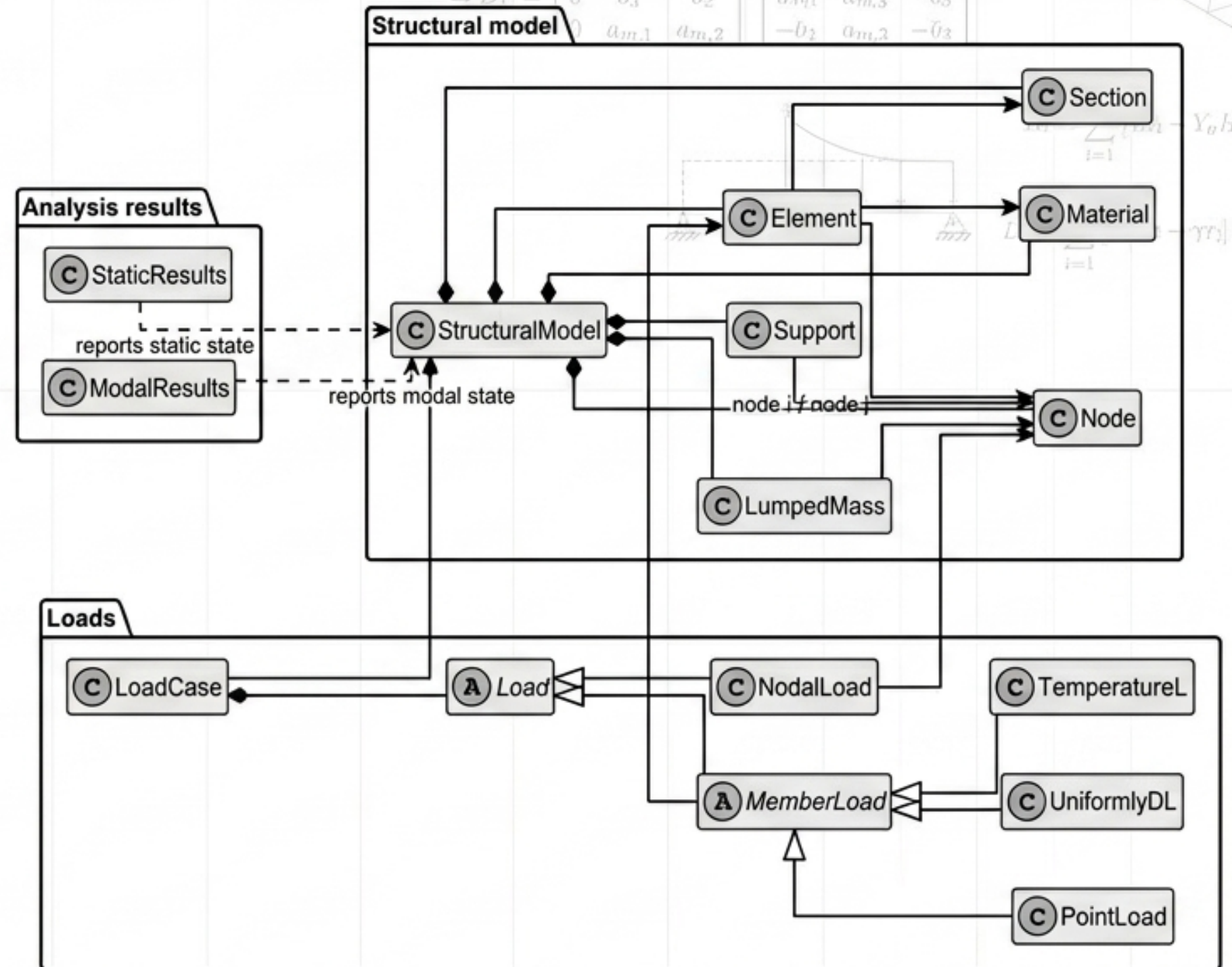


Domain Classes Reflect Standard Structural Mechanics

The Hub: StructuralModel contains all Nodes, Elements, Materials, Sections, Supports, and Loads.

Hierarchy & Math: Elements utilize standard local-to-global coordinate mappings ($q_l = T q_g$).

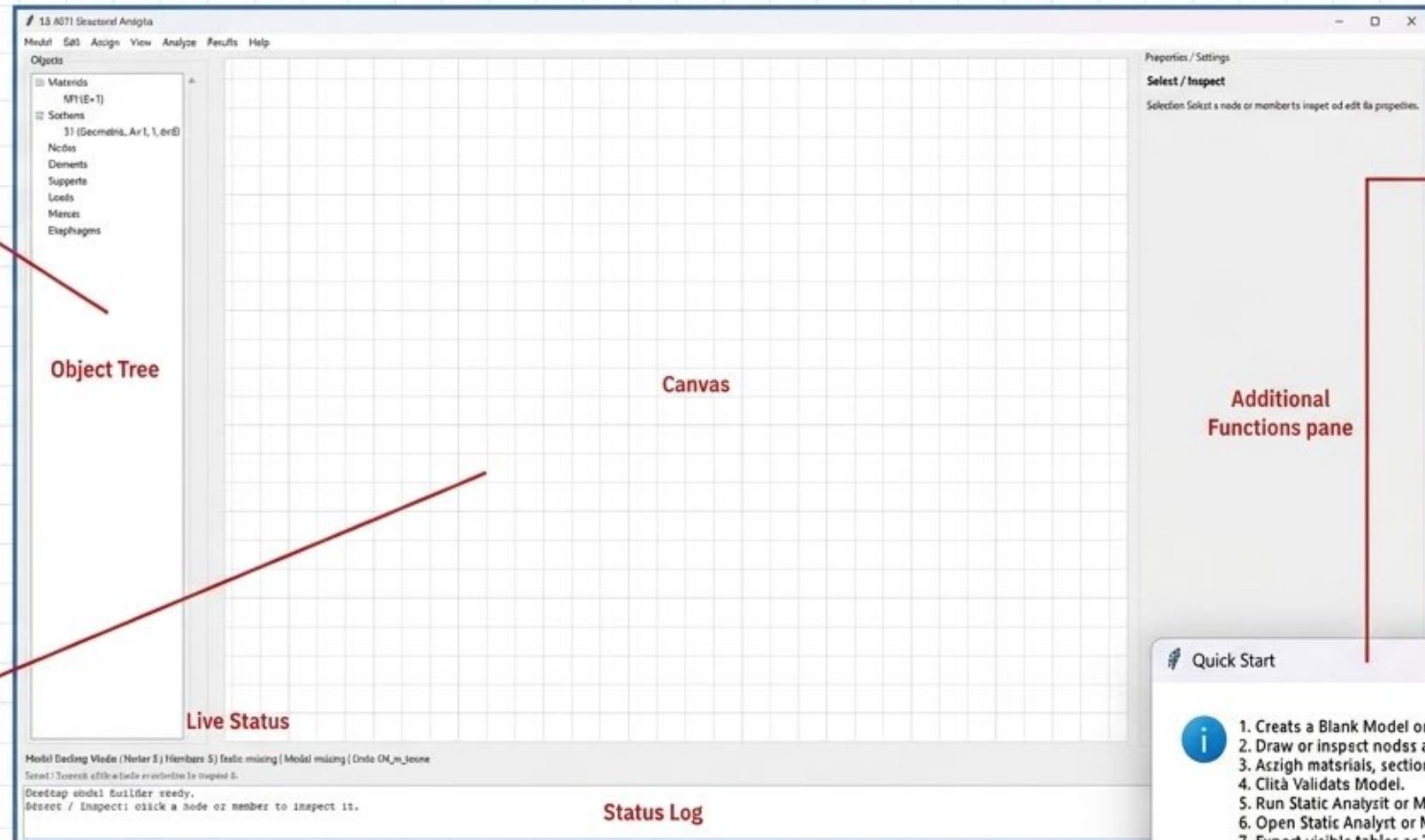
State Capture: Cleanly separated StaticResults and ModalResults classes freeze the completed analysis state.



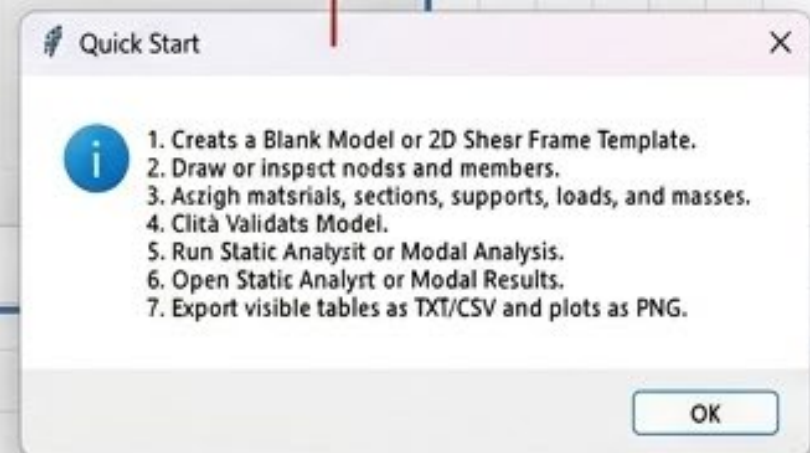
Interactive Desktop Model Building and Inspection

Object Tree:
Programmatic
editing of
loads, masses,
and boundary
conditions
without
writing code.

Canvas:
Interactive
node/element
drawing and
real-time
inspection.



Quick Start:
Blank canvas
initiation or
automated 2D
Shear Frame
templates.

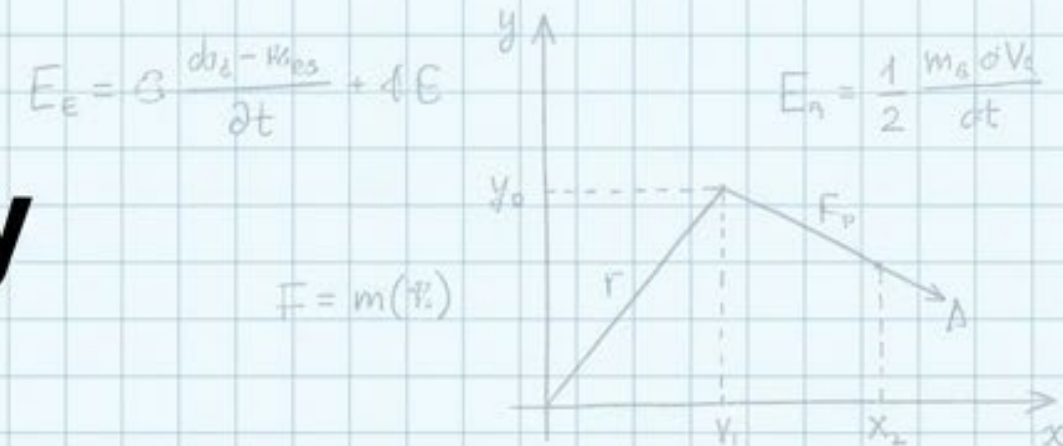


Rigorous Verification Against SAP2000 Benchmarks

Verification Strategy: Automated regression tests comparing XML-loaded models against SAP2000 text outputs and flexible-beam reference math.

Benchmark Case	Quantities Checked	Validation Status
Static Frame	Validates global assembly, displacements, reactions, and end-forces.	SAP2000 [PASS]
Settlement	Validates prescribed support displacements ($K_{fr} * u_r$).	SAP2000 [PASS]
Temperature	Validates thermal fixed-end effects and inclined truss transformations.	SAP2000 [PASS]
Modal Flexible-Beam	Validates condensed stiffness, frequencies, and mass normalization.	Reference [PASS]

Complete System Stability Backed by Automated Testing



Final Stable Baseline

197 total passed tests (pytest tests/ -q).

Focused Coverage Areas

- **63 UI/Static Tests:** XML parsing, ModelBuilder, direct stiffness, mass condensation.
- **76 Visualizer/Export Tests:** Missing-field tolerances, headless desktop export paths.

```
Terminal
(.venv) PS C:\Users\WUWUWU\Documents\GitHub\CE4011-Final-Project-V2> pytest tests/
platform win32 -- Python 3.14.3, pytest-9.0.3, pluggy-1.5.0
rootdir: C:\Users\WUWUWU\Documents\GitHub\CE4011-Final-Project-V2
plugins: anyio-4.13.0
collected 197 items

tests\test_controller.py ..... [ 28%]
tests\test_dynamic_assembly.py ..... [ 65%]
tests\test_dynamics.py ..... [ 165%]
tests\test_interface.py ..... [ 125%]
tests\test_modal_solver.py ..... [ 165%]
tests\test_model.py ..... [ 145%]
tests\test_model_builder.py ..... [ 145%]
tests\test_regression.py ..... [ 285%]
tests\test_response_spectrum.py ..... [ 225%]
tests\test_sap2000_strict.py ..... [ 245%]
tests\test_rtafc.py ..... [ 295%]
tests\test_time_history.py ..... [ 425%]
tests\test_ui_desktop_static.py ..... [ 465%]
tests\test_ui_dynamics.py ..... [ 775%]
tests\test_ui_modal_laport.py ..... [ 865%]
tests\test_ui_results.py ..... [ 825%]
tests\test_ui_shell.py ..... [ 845%]
tests\test_ui_stats.py ..... [ 975%]
tests\test_ui_visualization.py ..... [ 975%]
tests\test_unit.py ..... [ 955%]
tests\test_visualizer.py ..... [ 965%]
tests\test_visualizer_integration.py ..... [ 1000%]

===== 197 passed in 2.19s =====
(.venv) PS C:\Users\WUWUWU\Documents\GitHub\CE4011-Final-Project-V2>
```


Limitations, Future Work, and Conclusion



Current Limitations

- Exclusively 2D (u_x, u_y, r_z).
- Utilizing an educational standard-library numerical core.



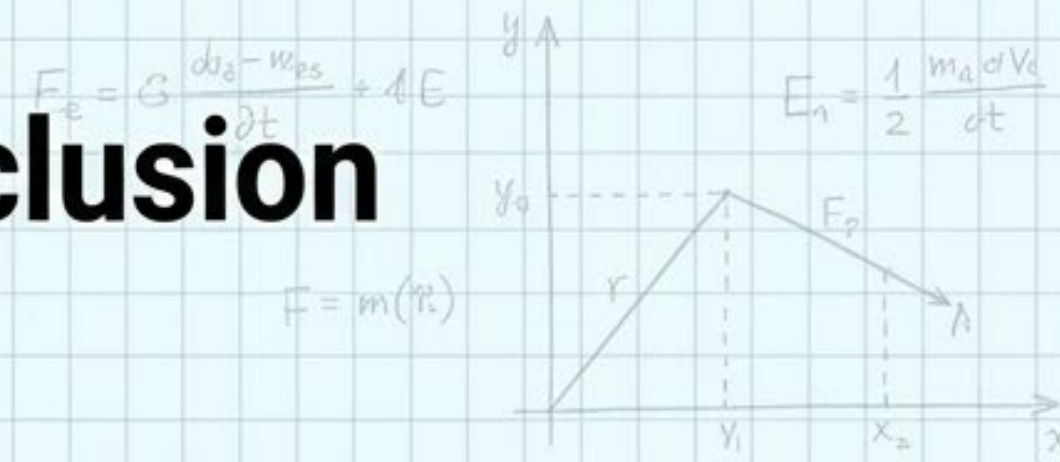
Future Extensions

- Promote deferred RSA and THA backends to desktop workflows.
- Implement 3D transformations, rotational springs, and automated reporting.



Conclusion

Delivered a highly stable, transparent, object-oriented MVP for two-dimensional Static and Modal analysis that successfully fulfills the CE 4011 project requirements.



$$E = \frac{1}{R} \frac{d^2 u}{dx^2}$$



$$Z = Ux + (r_z)^2$$

$$V = \frac{K_r \cdot U_z^2}{2r}$$

$$\sum P_{xi} = C_{max} \times \frac{2(A_y + l_i)^2}{2r^2}$$

$$W_{lib} = \frac{1}{n} \cdot A_0 (U_R R_{20} + A_1)$$